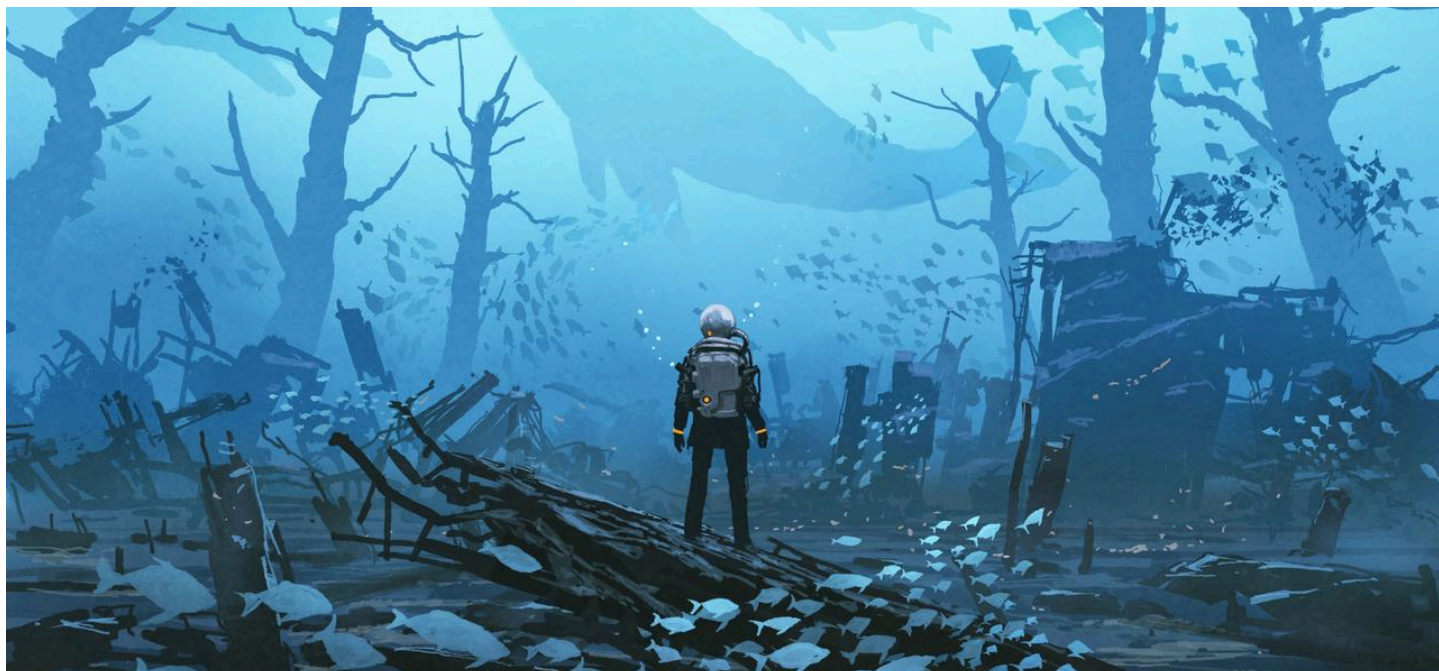




3-while

1. 简介



本节介绍循环结构——`while` 与 `do-while` 语句，及其背后的"循环三要素"。另外，当需要存储、处理一组数据时，按需引入 `std::string`、`std::vector` 与 `std::array` 容器。

学习目标：学完本节后，你应该能够：

- 理解循环的作用：让程序重复执行某段代码；
- 掌握 `while` 循环的语法与执行流程，理解"循环三要素"（初始化、条件判断、变量更新）；
- 掌握 `do-while` 循环的语法，理解它与 `while` 的核心区别——至少执行一次；
- 在循环中配合 `if` 与 `break` 控制流程；
- 初步了解 `std::string`、`std::vector` 与 `std::array`，用 `while` 遍历其中的数据。

本节重点：

- `while` 与 `do-while` 的语法与执行流程；
- 循环条件的写法与循环变量的更新；
- `break` 语句提前退出循环。

本节难点：

- 识别循环结构与循环条件——`while` 循环往往由"条件驱动"，而非固定次数；
- 避免死循环：确保循环条件最终为假；
- `do-while` “至少执行一次”的特性；
- 从“处理一个数”到“处理一组数”的思维过渡。

2. 循环与循环不变式

程序设计语言的循环结构用于表达“重复做某件事”。C++ 提供了两种循环结构，即 `for` 循环与 `while` 循环。它们的表达能力是等价的，也各有其最适合的使用场合。本节先介绍 `while` 循环。

2.1 `while` 循环

在动手写代码之前，先介绍 `while` 循环的语法（参见 [while 语句 @ cppreference](#)）：

代码块

```
1 while ( condition ) statement
```

只要条件 `condition` 为真，就反复执行循环体 `statement`。写 `while` 循环时，需要关注三个要素——我们称之为**循环三要素**：

- **初始化**：在进入循环前，把相关变量设置为初始状态；
- **条件判断**：`while` 后的条件决定循环是否继续；
- **变量更新**：循环体中的语句让变量的值不断变化，最终使条件为假。

如果条件永远为真，循环就永远不会结束，形成**死循环**。写循环时务必确认条件最终会变为假。

2.2 循环不变式

对于初学者而言，循环难以理解，是因为程序状态始终在变。然而，理解循环，最重要的恰恰是抓住变化中不变的本质，称为循环不变式。

 **循环不变式**（loop invariant）：在循环的**每次迭代开始之前**都成立的性质，是论证循环正确性的核心工具。要证明一个循环正确，只需证明三件事：

- **初始化**：进入循环之前，性质成立；
- **保持**：若某次迭代开始前性质成立，则这次迭代结束后（即下一次迭代开始前）性质仍然成立；
- **终止**：循环结束时，由性质可以推出我们所求的结果。

理解循环不变式，关键在于分清两件事：变量值在每一轮迭代中都在变化，不变的只是变量之间（或变量与输入之间）的某个**关系**。因此，每分析一个循环，都可以先问自己：



What remains true before every iteration?

每次迭代开始之前，什么性质始终成立？

找到这个性质后，再按“初始化—保持—终止”三步验证，就可以确信循环结束时得到的是正确结果。

2.3 循环不变式作为一种抽象

循环不变式首先是一个关于程序状态的性质；从程序理解和证明的角度看，它把循环执行过程中大量不同的具体状态概括为一个稳定不变的性质。它是一种**语义抽象**（reasoning abstraction），其要义是：舍弃无关的具体细节，只保留与正确性有关的结构。

以 `euclid.cpp`（3.3 节）为例（建议学习完第 3 节后再回头来看）：输入 `48 18` 时，`(a, b)` 依次经历 `(48, 18) → (18, 12) → (12, 6) → (6, 0)`。不加抽象，我们面对的是四个具体状态和若干次赋值；循环不变式则用一句话概括它们：



`gcd(a, b) = gcd(a0, b0)`

即当前这对 `(a, b)` 仍然表示同一个最大公约数问题。我们不再关心 `a`、`b` 具体是几、已经执行了几轮——这些与正确性无关的细节被抽象掉了。这一抽象体现在三个层面：

- **状态抽象**：大量具体状态 → 一个不变的性质；
- **时间抽象**：任意多次迭代 → 一次推理（初始化—保持—终止）；
- **推理抽象**：整段循环 → 一个高层操作（而不是一堆赋值语句）。

循环隐藏任意多次的重复执行、暴露不变式。发现不变式本身也是一个抽象过程：先执行跟踪（手工跑几轮），再发现规律（每轮开始时什么关系始终成立？），最后抽象表达（把规律写成一句话或一个公式）。

3. `euclid`

输入两个正整数，求它们的最大公约数。

两个正整数的最大公约数（Greatest Common Divisor, GCD）是能同时整除它们的最大正整数。例如 `gcd(48, 36) = 12`。

3.1 Euclid Algorithm

数学上有一个优雅的最大公约数算法——辗转相除法（[欧几里得算法](#)），它基于如下事实：



$$\text{gcd}(a, b) = \text{gcd}(b, a \% b)$$

也就是说， a 与 b 的最大公约数，等于 b 与 a 除以 b 的余数的最大公约数。反复套用这一等式，余数不断变小，最终会变成 0，此时另一个数就是最大公约数。例如： $\text{gcd}(48, 36) = \text{gcd}(36, 12) = \text{gcd}(12, 0) = 12$ 。

3.2 识别 loop

辗转相除的过程正是一个典型的循环：

循环体：计算余数 $r = a \% b$ ，然后令 $a = b$ 、 $b = r$ ；

- **循环条件：** $b \neq 0$ ，即“还有余数需要处理”。

循环结束时 $b == 0$ ，此时 a 就是最大公约数。注意这里的循环条件不是“数到某个固定次数”，而是由数据本身决定的——这正是 `while` 循环擅长解决的问题。

3.3 euclid

euclid.cpp

```
1 // File: euclid.cpp
2 #include <iostream>
3
4 int main() {
5     int a{};
6     int b{};
7     std::cin >> a >> b;
8
9     while (b != 0) {
10         int r{a % b};
11         a = b;
12         b = r;
13     }
14
15     std::cout << "gcd: " << a << '\n';
16     return 0;
17 }
```

样例输入 / 预期输出

```
1 48 36 => gcd: 12
2 17 5  => gcd: 1
```

- `while (b != 0)`：循环条件是“还有余数”——只要还有余数，就继续辗转相除。读者不妨对比后文数位个数问题中“还有数字”（`n > 0`）的循环条件，体会其中共通的思想。
- 循环体中的更新顺序至关重要：必须先计算余数 `r`，再依次更新 `a` 与 `b`。如果先修改了 `a`，余数就算错了。
- 循环结束时 `b == 0`，`a` 即为最大公约数。读者不妨用 48 与 36 手工模拟一遍循环的每次迭代，体会“子问题规模不断缩小”的过程。

3.4 Loop Invariant

现在用循环不变式的框架来分析 `euclid.cpp`。这个循环最核心的不变式是：



`gcd(a, b) = gcd(a0, b0)`

其中 `a0`、`b0` 是循环开始前的原始输入。它成立的依据是 3.1 节的 Euclid 恒等式：循环体把状态 `(a, b)` 变成 `(b, a % b)`，而 `gcd(a, b) = gcd(b, a % b)`，最大公约数保持不变。

按“初始化—保持—终止”三步验证：

初始化：进入循环前 `a = a0`、`b = b0`，`gcd(a, b) = gcd(a0, b0)` 平凡成立；

保持：设某次迭代开始前 `gcd(a, b) = gcd(a0, b0)`。循环体令 `a' = b`、`b' = a % b`，由 Euclid 恒等式 `gcd(a', b') = gcd(b, a % b) = gcd(a, b) = gcd(a0, b0)`，不变式保持；

终止：循环结束时 `b == 0`。对 `a ≥ 0` 有 `gcd(a, 0) = a`，故 `a = gcd(a0, b0)`——程序输出的正是最大公约数。

这个循环的**终止性**也一目了然：当 `b ≠ 0` 时，`0 ≤ a % b < b`，新的 `b` 严格小于旧的 `b`；非负整数不可能无限严格递减，因此循环必然结束。整个算法可以浓缩为两句话：



Loop invariant: `gcd(a, b) = gcd(a0, b0)` .

Termination measure: `b` is a nonnegative integer that strictly decreases whenever the loop continues.

用输入 48 18 验证：`gcd(48, 18) = gcd(18, 12) = gcd(12, 6) = gcd(6, 0) = 6`，每一步都满足不变式。

3.5 euclid-bruteforce

另一种思路是：从 `min(a, b)` 开始，逐个向下尝试，找到第一个能同时整除 `a` 与 `b` 的数。

`euclid-bruteforce.cpp`

```
1 // File: euclid-bruteforce.cpp
```

```

2  #include <algorithm>
3  #include <iostream>
4
5  int main() {
6      int a{};
7      int b{};
8      std::cin >> a >> b;
9
10     int d{std::min(a, b)};
11     while (d >= 1) {
12         if (a % d == 0 && b % d == 0) {
13             break;    // d is a common divisor
14         }
15         --d;
16     }
17
18     std::cout << "gcd: " << d << '\n';
19     return 0;
20 }

```

样例输入 / 预期输出

```

1  48 36 => gcd: 12
2  17 5  => gcd: 1

```

`int d{std::min(a, b)};` 使用 C++ 标准库函数 `std::min` 初始化 `d`：它返回两个参数中的较小者。`std::min` 在头文件 `<algorithm>` 中声明（参见 [std::min @ cppreference](#)）。

`<algorithm>` 是标准库中与“算法”相关的头文件，包含大量通用算法函数，我们会在后续章节中反复用到。

循环体中第一次遇到 `break` 语句。`break` 的作用是**立即终止当前所在的循环**，程序跳到循环之后继续执行。循环从 `d = min(a, b)` 开始，从大到小逐个尝试：如果 `d` 能同时整除 `a` 与 `b`，它就是最大公约数，继续试更小的数没有意义，用 `break` 立刻退出循环；否则把 `d` 减 1 继续试。

 请思考，如果没有第 13 行的 `break`，会导致什么问题？

与上面的 `euclid.cpp` 相比，这个朴素版本效率不高：在最坏情况下（例如 `gcd(1, 1000000007)`），循环要从 `1000000007` 一直试到 `1`，需要约十亿次尝试；而辗转相除法每次迭代都把余数大幅缩小，只需几十次。

3.6 euclid-while-decl

在 `euclid-bruteforce.cpp` 中，`d` 在进入循环之前声明并初始化。能不能把 `d` 的声明也"放进" `while` 语句本身？这要用到 `while` 语法的一个细节：条件（condition）可以是简单声明（simple declaration）。



cppreference 对 `while` 语句条件的说明（[while 语句 @ cppreference](#)）：

A condition can either be an [expression](#) or a [simple declaration](#). If condition is a simple declaration, the value it yields is the value of the decision variable (see below) contextually converted to `bool`.

也就是说，条件可以直接写成 `int d{...}` 这样的声明：`d` 称为决策变量（decision variable），条件是否为真，取决于 `d` 的值经上下文转换（contextual conversion）为 `bool` 后的结果——对于整数，这就是 `d != 0`。

按这个思路改写 `euclid-bruteforce.cpp`：把 `d` 的声明放进 `while` 的条件，并把 `cout` 行移动到 `break` 之前：

`euclid-while-decl-wrong.cpp`

```
1 // File: euclid-while-decl-wrong.cpp
2 #include <algorithm>
3 #include <iostream>
4
5 int main() {
6     int a{};
7     int b{};
8     std::cin >> a >> b;
9
10    while (int d{std::min(a, b)}) {
11        if (a % d == 0 && b % d == 0) {
12            std::cout << "gcd: " << d << '\n';
13            break;
14        }
15        --d;
16    }
17
18    return 0;
19 }
```

先看"等价"成立的一面：从条件本身来说，`(bool)d` 即 `d != 0`；由于 `d` 从正整数 `std::min(a, b)` 出发、只减不增，它始终非负，因此 `d != 0` 与 `d >= 1` 的真值完全相同。在这一层意义上，`while (int d{...})` 与原来的 `while (d >= 1)` 是等价的。

但程序的整体行为并不等价：条件中的声明在每次迭代都会重新创建并初始化——[cppreference](#) 明确说明 "the variable it declares is destroyed and created with each iteration of the loop"。也就是说，每轮求值条件时 `d` 都被重新初始化为 `std::min(a, b)`，循环体里的 `--d` 对下一轮判断完全无效；只要 `std::min(a, b) >= 1`（本题输入为正整数，恒成立），条件就永远为真，程序陷入死循环。

 **实测验证：**用 GCC 16（`-std=c++20`）编译运行上面的代码、输入 `48 36`，程序不输出任何结果、也无法终止（死循环）。把 `d` 的声明放回循环之外（3.5 节的写法），或改用 `for` 语句（`for (int d = std::min(a, b); d >= 1; --d)`，参见 [for 语句 @ cppreference](#)，下一讲《4-for》的主角），才是正确的做法。

条件声明适合"决策变量每轮重新求值"的场景，例如 `while (char c = read_char())`（每轮读取一个新字符并判断）；而"先初始化、循环体内更新、再判断"的模式需要变量在循环外保持状态，不应把声明写进条件。

错误示例：while 没有初始化器

```
1 while (int d{std::min(a, b)}; d >= 1) { // syntax error: while has no init-  
    statement  
2     // ...  
3 }
```

顺带澄清一个常见误解：`while (int d{...}; d >= 1)` 这种带分号的写法**不是合法 C++**。带初始化器 (init-statement) 的语法只有 `if` 与 `switch` 才有——自 C++17 起可以写 `if (init; condition)`、`switch (init; condition)`（参见 [if 语句 @ cppreference](#)），但 `while` 没有这种形式。用 GCC 16 编译上面的写法会得到语法错误 `expected ')' before ';'`。

为什么 `while` 不像 `if` / `switch` 那样提供初始化器？因为 `for` 语句已经完整覆盖了这个需求：`for (init; condition; update)` 把"初始化、条件、更新"三部分集中在一个语句里；若 `while` 再提供初始化器，会与 `for` 功能重复。语言设计者刻意让 `while` 保持"只关心条件"的简洁定位。

4. fib

输入一个非负整数 `n`，输出第 `n` 个斐波那契数 `F(n)`。

斐波那契数 (Fibonacci numbers) 是数学中一个著名的数列，其定义采用英文 Wikipedia ([Fibonacci number @ Wikipedia](#))：

 $F(0) = 0$ ， $F(1) = 1$ ，且对 $n > 1$ ， $F(n) = F(n-1) + F(n-2)$ 。

也就是说，从第 2 项开始，每一项都等于前两项之和。数列开头为：0, 1, 1, 2, 3, 5, 8, 13, 21, 34, ...

定义中的递推（recurrence）思想是：要算第 n 项，先算第 $n-1$ 项与第 $n-2$ 项；而这两项又依赖更小的项……一直追到 $F(0)$ 、 $F(1)$ 这两个已知的起点。

4.1 识别 loop

顺着递推关系，我们可以从两个已知数出发，一步步“推”出后面的数：先有 $F(0) = 0$ 与 $F(1) = 1$ ，两者相加得到 $F(2) = 1$ ；再用 $F(1)$ 、 $F(2)$ 得到 $F(3) = 2$ ……这个过程本身就是一个循环：

- **循环体**：计算下一个数（前两项之和），并把“窗口”前移一格；
- **循环条件**：还需要推进一步，即 $\text{index} < n$ （目前已得到第 index 项）。

对照循环三要素：

- 初始化—— $\text{previous} = F(0) = 0$ ， $\text{current} = F(1) = 1$ ， $\text{index} = 0$ ；
- 条件判断—— $\text{index} < n$ ；
- 变量更新——每次迭代把 previous 、 current 推进为相邻的后两项，并让 index 加 1。

4.2 fib

fib.cpp

```
1 // File: fib.cpp
2 // Compute the nth Fibonacci number.
3 // We assume that  $n \geq 0$  and all intermediate results fit in int.
4
5 #include <format>
6 #include <iostream>
7
8 int main() {
9     int n{};
10    std::cin >> n;
11
12    int previous{0}; //  $F(0)$ 
13    int current{1};  //  $F(1)$ 
14    int index{0};
15
16    while (index < n) {
17        int next{previous + current};
18        previous = current;
19        current = next;
20        ++index;
21    }
```

```

22
23     std::cout << std::format("F({}) = {}\n", n, previous);
24
25     return 0;
26 }

```

样例输入 / 预期输出

```

1  0 =>  F(0) = 0
2  1 =>  F(1) = 1
3  7 =>  F(7) = 13

```

下面再从几个角度进一步理解这段代码：

- **窗口与临时变量：** `previous`、`current` 始终是相邻的两个斐波那契数；先算 `next = previous + current`，再整体前移，避免先覆盖 `previous` 导致 `current` 算错。这正是上面循环不变式所刻画性质。
- **边界情况：** `n = 0` 时循环一次也不执行，`previous` 保持 `F(0) = 0`，程序直接输出 `0` —— 正确。这正体现了 `while` 循环“可能一次都不执行”的特性。
- **计数驱动：** `index` 从 0 递增到 `n`，循环次数由输入 `n` 决定，是“计数驱动”的循环。
- **溢出：** `int` 能表示的斐波那契数有限，`F(46) = 1836311903` 已经是 `int` 能表示的最大一项，`F(47)` 就会溢出。需要更大的 `n` 时，把 `previous`、`current`、`next` 的类型换成 `long long` 即可，其余代码不变。

4.3 Loop Invariant

对于 `fib.cpp`，循环不变式可以表述为：



At the beginning of each iteration, `previous == F(index)` and `current == F(index + 1)`. Each iteration advances this pair by one position in the Fibonacci sequence.

每次循环开始时，`previous` 保存 `F(index)`，`current` 保存 `F(index+1)`。一次迭代将这对相邻的斐波那契数向前推进一个位置。

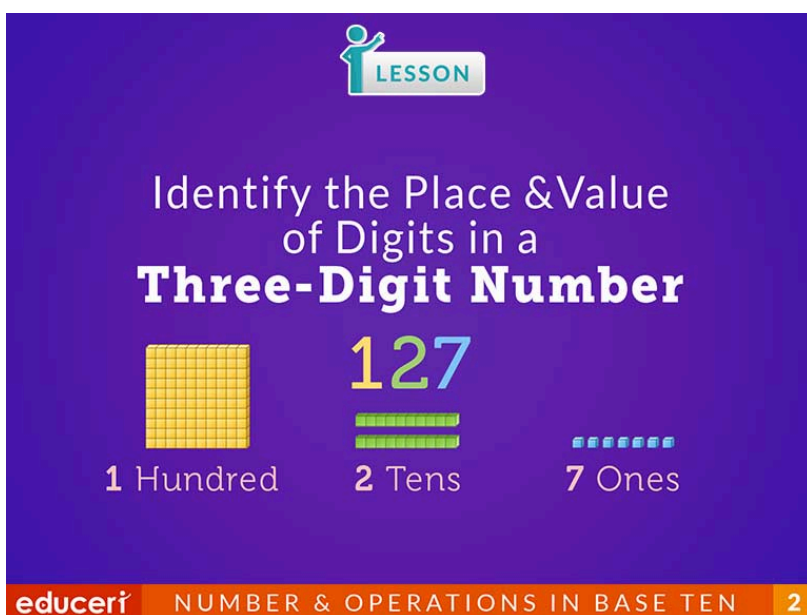
按“初始化—保持—终止”三段验证：

- **初始化：** 进入循环前 `index = 0`，`previous = 0 = F(0)`，`current = 1 = F(1)`，不变式成立；

- **保持：**假设某次迭代开始前 `previous = F(index)`、`current = F(index+1)`。执行循环体后，`next = previous + current = F(index+2)`，随后 `previous = current = F(index+1)`、`current = next = F(index+2)`，且 `index` 递增 1——下一次迭代开始前不变式依然成立；
- **终止：**`index` 每轮递增 1，条件 `index < n` 最终必然为假，循环结束时 `index == n`；由不变式得 `previous = F(n)`，程序输出的正是第 `n` 个斐波那契数。

5. digits-do-while

计算给定的非负整数的位数。假设 0 是 1 位数。



5.1 数位个数：while 版本与边界处理

本节的问题是：求一个非负整数的位数（假设 0 是 1 位数）。先用 `while` 循环写一版（文件 `digits-while.cpp`）。注意：当 `n = 0` 时，循环体一次也不会执行，而 0 是 1 位数——这个边界情况需要在循环之外特殊处理。

digits-while.cpp

```
1 // File: digits-while.cpp
2 #include <iostream>
3
4 int main() {
5     int n{};
6     std::cin >> n;
7
8     int count{0};
9     if (n == 0) {
10         count = 1; // special case: 0 has 1 digit
11     } else {
12         while (n > 0) {
```

```

13         ++count;
14         n /= 10;
15     }
16 }
17
18 std::cout << "Number of digits: " << count << '\n';
19 return 0;
20 }

```

样例输入 / 预期输出

```

1  0  =>  Number of digits: 1
2 123 =>  Number of digits: 3

```

这个版本是正确的：输入 `0` 时走 `if` 分支输出 `1`，输入 `123` 时循环三次输出 `3`。但请注意，为了处理 `0` 这个边界情况，我们不得不在循环外加一个 `if` 分支，把逻辑分成两条路径——代码因此变得繁琐。

为什么需要特判？因为求位数时，即使 `n = 0`，我们也至少需要执行一次循环体（先把 `count` 加 1）；而 `while` 循环先判断后执行，可能一次都不执行，两者的语义不匹配。如果一个循环**先执行一次循环体、再判断条件**，就不需要这个特判了——这正是 5.2 节 `do-while` 循环的用武之地。

5.2 do-while 循环

`do-while` 循环与 `while` 循环的唯一区别是判断时机：它先执行一次循环体，再判断条件；只要条件为真，就继续执行。

digits-do-while.cpp

```

1  // File: digits-do-while.cpp
2  #include <iostream>
3
4  int main() {
5      int n{};
6      std::cin >> n;
7
8      int count{0};
9      do {
10         ++count;
11         n /= 10;
12     } while (n > 0);
13
14     std::cout << "Number of digits: " << count << '\n';
15     return 0;

```

```
16 }
```

样例输入 / 预期输出

```
1 0 => Number of digits: 1
2 12345 => Number of digits: 5
```

对 `n = 0`：循环体先执行一次，`count` 变为 1、`n` 变为 0，然后判断条件 `n > 0` 为假，循环结束，输出 1。正确！注意 `do-while` 语句末尾有分号 `;`，不要漏掉。

5.3 `while` 与 `do-while` 的对比

特性	<code>while</code>	<code>do-while</code>
判断时机	先判断，后执行	先执行，后判断
最少执行次数	0 次	1 次
适用场景	可能一次都不执行	至少执行一次
语法	<code>while (条件) { ... }</code>	<code>do { ... } while (条件);</code>

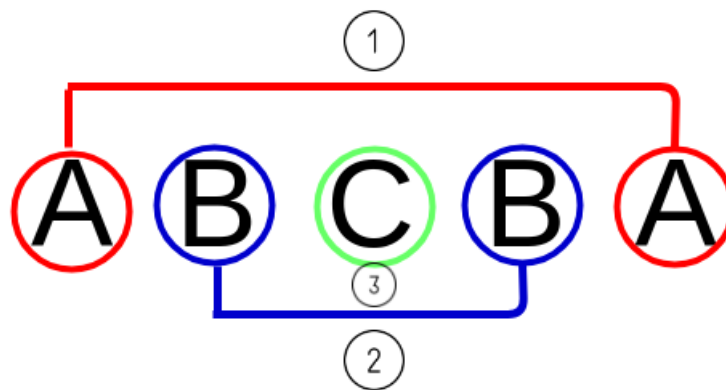
一般来说，能用 `while` 表达的问题也都能用 `do-while` 表达，反之亦然。选择哪一种，取决于问题是否需要“至少执行一次”。

 C++ Core Guidelines 对这个问题给出了一条明确建议 ([ES.75: Avoid do-statements](#))：优先使用 `while`，而不是 `do-while`。

理由有两点：可读性——终止条件位于循环末尾，容易被读者忽略；防错——条件在第一次进入循环时不会被检查，可能造成“本不应执行却执行了一次”的隐蔽错误。当然，该规则也承认确实存在 `do-while` 更自然的场景（例如“先读入、后判断”），只是此类场景远少于它带来的潜在问题。因此，只有“至少执行一次”确属问题本身的要求时，才值得使用 `do-while`；默认应首选 `while`。

6. `palindrome`

判定给定的字符串是否为回文串。



回文串 (palindrome) 是正读与反读相同的字符串，例如 "12321"、"abba" 都是回文串，而 "12345" 不是。如何判断一个字符串是否为回文串？最自然的想法是：从两端向中间逐对比较字符，只要发现某一对字符不相等，就可以断定它不是回文串。比较的过程用 `while` 循环表达：用两个下标 `left` 与 `right` 分别指向字符串的两端，每次比较后 `left` 向右移一位、`right` 向左移一位。

6.1 `std::string`

要逐个访问字符，我们使用 `std::string` 类型（参见 [std::string @ cppreference](#)）来保存输入的字符串。它需要包含头文件 `<string>`，并具有两个与本节相关的成员函数：

- `s.size()`：返回字符串中字符的个数；
- `s.at(i)`：返回下标 `i` 处的字符；与下标运算符 `s[i]` 相比，`at` 在越界时会抛出异常，更安全，初学阶段建议统一使用 `at`。

6.2 `palindrome`

确定比较范围：`left` 从 0 开始，`right` 从 `s.size() - 1` 开始。只要 `left < right` 就继续比较（两指针相遇或交错时，说明所有相对位置的字符都已比较完毕）。

palindrome.cpp

```
1 // File: palindrome.cpp
2 #include <format>
3 #include <iostream>
4 #include <string>
5
6 int main() {
7     std::string s{};
8     std::cin >> s;
9
10    std::size_t left{0};
11    std::size_t right{s.size() - 1}; // s is not empty, so no underflow here
12    bool isPalindrome{true};
13
```

```

14     while (left < right) {
15         if (s.at(left) != s.at(right)) {
16             isPalindrome = false;
17             break;
18         }
19         ++left;
20         --right;
21     }
22
23     if (isPalindrome) {
24         std::cout << std::format("{} is a palindrome.\n", s);
25     } else {
26         std::cout << std::format("{} is not a palindrome.\n", s);
27     }
28
29     return 0;
30 }

```

样例输入 / 预期输出

```

1  abcba =>  abcba is a palindrome.
2  abc   =>  abc is not a palindrome.

```

下面从几个角度进一步理解这段代码：

- `std::size_t` 是无符号整数类型，通常用于表示大小和下标。这里 `left` 与 `right` 都不会为负，用 `std::size_t` 是合适的。
- 循环条件 `left < right`：当两个下标相遇（`left == right`）或交错（`left > right`）时，说明已经比较完所有需要比较的字符对，循环结束。
- 一旦发现字符不相等，用 `break` 提前退出循环。
- 注意 `right = s.size() - 1`：如果输入是空字符串，`s.size()` 为 0，`s.size() - 1` 会因无符号下溢而变成一个极大的数。本题输入约定为非空字符串，因此不会出现这个问题。

7. binary-search

给定一个有序的（假设从小到大，允许有重复值）整数数组 `dict`，要求从中查找某个整数 `key`，返回该整数所在的下标。如果 `key` 不在 `dict` 中，则报告“未找到”。

7.1 数组

与前面的问题不同，本题要处理一组数据，而不是单个变量。为此，我们需要一种新的数据结构——数组。C++ 中的数组（[array @ cppreference](#)）是一块连续存储、类型相同的元素集合。

7.1.1 数组中的数据

数组中的数据存储在一片连续的内存区域中。声明数组时要同时指定元素类型与大小，例如 `int numbers[5]{23, 56, 19, 11, 78}` 声明了一个包含 5 个整数的数组，其类型为 `int [5]`。注意，数组的类型包含大小信息：`int [3]` 与 `int [5]` 是两种不同的类型（这一点在以后学习指针时会更清楚）。

数组元素通过一对 `{}` 初始化（统一初始化，参见 [list initialization @ cppreference](#)）。如果 `{}` 中的元素个数少于 `[]` 指定的大小，剩余元素被零初始化；多于指定大小则产生编译错误：

代码块

```
1  int numbers[5]{23, 56, 19}; // numbers = {23, 56, 19, 0, 0}
2  int numbers[5]{};          // numbers = {0, 0, 0, 0, 0}
3  int numbers[5]{1};         // numbers = {1, 0, 0, 0, 0}, not {1, 1, 1, 1, 1}
```

空的花括号 `{}` 表示值初始化（value initialization），将所有元素置 0。如果 `[]` 中不写大小，则由编译器根据初始化列表的元素个数推导：

代码块

```
1  int numbers[]{23, 56, 19, 11, 78}; // size of numbers is 5
```

7.1.2 数组上的操作

数组元素通过[数组下标访问运算符](#)（array subscript operator）`[]` 访问，下标从 0 开始，而不是从 1 开始。可以把下标理解成距离数组首元素的偏移量（offset）。例如 `a[2] = 4` 将下标为 2 的元素改为 4，`int min{numbers[0]}` 取出第一个元素。

数组本身没有提供求大小的成员函数。可以用 [sizeof @ cppreference](#) 运算符计算：数组总字节数除以单个元素的字节数。

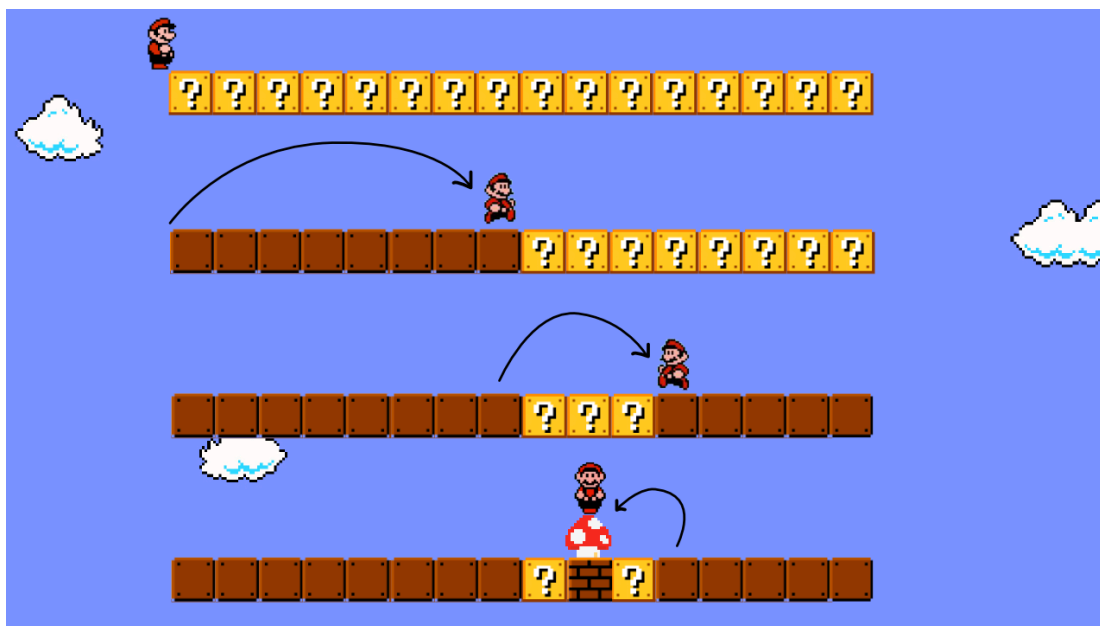
代码块

```
1  int numbers[]{23, 56, 19, 11, 78};
2  std::size_t size{sizeof(numbers) / sizeof(numbers[0])}; // size = 5
```

`sizeof` 作用在数组变量上时，结果是整个数组所占的字节数；作用在类型上时，结果是该类型变量所占的字节数。`sizeof(type)` 的括号不可省略，而 `sizeof(variable)` 的括号可以省略。它是运算符（不是函数），且在编译期确定结果，不涉及运行时的函数调用。

7.2 二分查找

如下图所示，假设超级玛丽要从一个大小为16（表示为 `[0 .. 15]`）有序（从小到大排序）的整数数组中查找数字 65（称为 key）。



他先检查数组中间（下标为 $(0 + 15) / 2 = 7$ ）的数字，假设为 50。因为数组是有序的且 $65 > 50$ ，左半边的数组 `[0 .. 7]` 中必定不存在要查找的数字。

问题就转化成了：在右边有序数组 `[8 .. 15]` 中查找数字 65。该问题与原问题本质相同，但是问题的规模缩小了一半。这提示我们可以使用同样的方法继续进行查找。超级玛丽继续检查有序数组 `[8 .. 15]` 中间（下标为 $(8 + 15) / 2 = 11$ ）的数字，假设为 80。因为 $65 < 80$ ，子数组 `[11 .. 15]` 中必定不存在要查找的数字。

此时，问题又转化成了：在子数组 `[8 .. 10]` 中查找数字 65。该问题仍然与原问题本质相同，但是问题的规模再次缩小了一半。

最后，超级玛丽检查有序数组 `[8 .. 10]` 中间（下标为 $(8 + 10) / 2 = 9$ ）的数字，假设恰好为 65。查找过程结束。

7.2.1 识别 loop

在上述查找过程中，有一个强烈的信号提示我们需要使用循环结构：经过一些操作，我们可以将原问题转化为本质相同、但规模更小的子问题。解决这些子问题是循环结构中每次迭代的任务，这些操作构成了循环结构的循环体，最小规模的子问题的结构则是构造循环条件的基础。

在二分查找过程中，子问题是从任意一段有序子数组中查找特定的元素。我们可以使用两个下标刻画一个子数组：`low` 表示子数组的左端，`high` 表示子数组的右端。进行的操作包括比较待查找元素与子数组中间元素的大小，并确定更小子问题所需考虑的子数组。最小规模的子问题只需从大小为 1 的数组中查找特定元素。也就是说，当 `low <= high` 时，循环继续。

7.2.2 binary-search

binary-search.cpp

```

1 // File: binary-search.cpp
2
3 #include <format>
4 #include <iostream>
5
6 int main() {
7     const int dict[]{1, 1, 2, 3, 5, 8, 13, 21, 34, 55};
8
9     int key{};
10    std::cin >> key;
11
12    int low{0};
13    int high{sizeof(dict) / sizeof(dict[0]) - 1};
14
15    int index{-1}; // -1: sentinel for "not found" (never a valid index)
16    while (low <= high) {
17        int mid{(low + high) / 2};
18
19        if (key > dict[mid]) {
20            low = mid + 1;
21        } else if (key < dict[mid]) {
22            high = mid - 1;
23        } else { // key == dict[mid]
24            index = mid;
25            break; // important!
26        }
27    }
28
29    if (index == -1) {
30        std::cout << "Not found!\n";
31    } else {
32        std::cout << std::format("The index of {} is {}.\\n", key, index);
33    }
34
35    return 0;
36 }

```

样例输入 / 预期输出

```

1 34 => The index of 34 is 8.
2 1  => The index of 1 is 1.
3 0  => Not found!

```

下面从几个角度进一步理解这段代码：

- Line 7 声明数组 `dict`： `const int dict[]{1, 1, 2, 3, 5, 8, 13, 21, 34, 55}`。这是 C 数组语法：元素类型为 `int`，长度由初始化列表推导为 10；`const` 表示数组内容只读——查找算法不修改数组，声明为 `const` 可防止误写。数组元素通过下标 `dict[mid]` 访问。
- Line 12 与 Line 13 分别声明变量 `low` 与 `high`，用于刻画待搜索的子数组。`low` 的初始值为 0，`high` 的初始值为 `sizeof(dict) / sizeof(dict[0]) - 1`，即数组长度减 1：`sizeof(dict)` 是数组占用的总字节数，`sizeof(dict[0])` 是单个元素占用的字节数，两者相除得到元素个数——这是 C 数组求长度的惯用法。这里统一使用带符号的 `int`（参见 [std::size_t @ cppreference](#)）：当 `mid` 为 0 且 `key` 小于中间元素时，需要执行 `high = mid - 1`，此时 `high` 应变为 -1、表示“区间已空”；带符号类型才能表示这个值。带符号下标也更贴近二分查找的标准写法（对应 C++ Core Guidelines ES.102：算术使用有符号类型）。
- Line 15 定义 `index` 变量，表示 `key` 在数组中的下标。`index` 的类型是带符号的 `int`，初始化为 -1：-1 不可能是合法的数组下标，正好用作表示“还没有找到”的哨兵值（sentinel），这也是许多查找类接口的惯用做法；如果在数组中找到了 `key`，则将 `index` 赋值为相应下标（见 Line 24）。在 Line 29 处，如果 `index == -1`，则表明 `key` 不在数组中。
- Lines 16--27 是循环体：先计算当前子数组中间元素的下标 `mid`（Line 17），然后根据 `key` 与 `dict[mid]` 的大小关系决定“扔掉”数组的左半边（Line 20）、“扔掉”数组的右半边（Line 22）还是在 `mid` 处找到了 `key`（Lines 23--26）。Line 16 指明了循环条件：`low <= high`。

Line 25 处的 `break` 语句不可少。假设 `key` 为 34。上述代码会在下标 8 处找到 `key` 值，也就是在 Line 24 处 `index` 更新为 8。如果没有接下来的 `break` 语句，循环将继续进行。但是，由于 `low` 与 `high` 都没有改变，循环条件依然成立，`mid` 也没有改变，程序仍然进入 Line 23 处的分支，从而导致无限循环。`break` 语句用于提前中止该循环。

7.2.3 binary-search-array

第 7.1 节介绍的数组仍是传统 C 语言中的数组，它没有提供便利的函数操作（比如求数组大小），对越界（out-of-bound）的下标访问也不进行任何检查。C++ 标准库提供了 [std::array @ cppreference](#)，它有如下优点：

- **定长数组**：`std::array<T, N>` 是元素类型为 `T`、长度为 `N` 的定长数组，定义在头文件 `<array>` 中。与 C 数组一样，它的长度在编译期固定；
- **成员函数**：[std::array::size @ cppreference](#) 函数返回数组大小。[std::array::at @ cppreference](#) 是带越界检查的下标访问，下标越界时抛出 `std::out_of_range` 异常，便于及时发现程序错误，而 C 数组的 `a[i]` 越界则是**未定义行为**；
- **非退化**：`std::array` 作为整体传递时保留大小信息，不会像 C 数组那样退化为指针（后续介绍函数时再展开介绍）。

```

1 // File: binary-search-array.cpp
2
3 #include <array>
4 #include <format>
5 #include <iostream>
6
7 int main() {
8     const std::array<int, 10> dict{1, 1, 2, 3, 5, 8, 13, 21, 34, 55};
9
10    int key{};
11    std::cin >> key;
12
13    int low{0};
14    int high{static_cast<int>(dict.size()) - 1};
15
16    int index{-1}; // -1: sentinel for "not found" (never a valid index)
17    while (low <= high) {
18        int mid{low + (high - low) / 2};
19
20        if (key > dict.at(mid)) {
21            low = mid + 1;
22        } else if (key < dict.at(mid)) {
23            high = mid - 1;
24        } else { // key == dict.at(mid)
25            index = mid;
26            break; // important!
27        }
28    }
29
30    if (index == -1) {
31        std::cout << "Not found!\n";
32    } else {
33        std::cout << std::format("The index of {} is {}.\n", key, index);
34    }
35
36    return 0;
37 }

```

样例输入 / 预期输出

```

1 34 => The index of 34 is 8.
2 1  => The index of 1 is 1.
3 0  => Not found!

```

接下来，简要介绍由于使用 `std::array` 带来的变化：

- Line 8 声明 `dict` : `const std::array<int, 10> dict{1, 1, 2, 3, 5, 8, 13, 21, 34, 55}`。与 C 数组版本相比：元素类型与长度作为模板参数 `<int, 10>` 显式给出。
- Line 14 用 `dict.size()` 获取元素个数，再减 1 得到最后一个元素的下标，取代了 C 数组的 `sizeof` 惯用法。由于 `size()` 返回无符号类型 `std::size_t`，这里用 `static_cast<int>(...)` 显式转换为带符号的 `int`。
- 循环体中的元素访问改用 `dict.at(mid)`： `at()` 在 `mid` 越界时抛出异常，便于调试；其余逻辑（`low` / `high` / `index` 的维护、`break` 的提前退出）不变。

7.2.4 算法复杂度

在最坏情况下，二分查找算法仅需约 $\log n$ 次比较操作。我们称其算法复杂度是对数（logarithmic）级别的。

 对于有序数组，二分查找算法是最优算法。你不妨试着证明一下，或许你能发展出一套证明策略。

需要注意的是，要说明二分查找算法的最优性，实际上是要说明有序数组查找问题本身固有的难度，而一个问题固有的难度与使用什么样的算法进行求解是无关的。因此，你的证明过程中不应出现任何特定的算法，你要考虑的是“所有可能的算法”。

7.3 二分查找变体

在 `binary-search.cpp` 中，如果查找 `key = 1`，则返回下标 `1`。`dict` 数组包含两个 `1`，如何修改代码让其返回下标 `0` 呢？更一般地，如果数组中存在多个 `key`，如何返回最左端（left-most）或者最右端（right-most）的 `key` 的下标？

一种直接的做法是在 Line 30 处添加代码，从 `index` 处继续向左扫描，查找最左端的 `key`。

向左扫描的朴素做法

```
1    // ...
2    if (index == -1) {
3        std::cout << "Not found!\n";
4    } else {
5        // continue scanning dict to the left
6        while (index > 0 && dict[index - 1] == key) {
7            --index;
8        }
9
10       std::cout << std::format("The index of {} is {}.\n", key, index);
11   }
12   // ...
```

然而，该做法在最坏情况下（比如，数组中全都是 `key` 值）需要进行约 $n / 2$ 次比较操作，又变成线性复杂度了。我们期望仍在对数级别复杂度约束下完成该任务。

`binary-search-leftmost.cpp` 与 `binary-search.cpp` 相比，只需要一处改动：找到 `key` 后不执行 `break`，而是继续向左搜索——把 Lines 23--26 中的 `break`（Line 25）换成 `high = mid - 1` 即可。乍一看，难以理解，然而以“原问题-子问题”的角度思考，一切豁然开朗：在 `mid` 处找到某个 `key` 之后，如果还想继续查找最左端的 `key`，则问题转化为“在 `[low .. mid - 1]` 子数组中继续查找 `key`”。该问题与原问题本质相同，故可以共用同一个 `while` 循环结构，只需要通过 `high = mid - 1` 将搜索范围限制在 `[low .. mid - 1]` 即可。当区间变为空（`low > high`）时循环自然结束，`index` 就是最左端的下标；即使 `mid` 为 0，`high = mid - 1` 也只是让 `high` 变为 -1、循环条件不再成立，无需任何特判。

`binary-search-leftmost.cpp`

```
1 // File: binary-search-leftmost.cpp
2
3 #include <array>
4 #include <format>
5 #include <iostream>
6
7 int main() {
8     const std::array<int, 10> dict{1, 1, 2, 3, 5, 8, 13, 21, 34, 55};
9
10    int key{};
11    std::cin >> key;
12
13    int low{0};
14    int high{static_cast<int>(dict.size()) - 1};
15
16    int index{-1}; // sentinel: not found
17    while (low <= high) {
18        int mid{low + (high - low) / 2};
19
20        if (key > dict.at(mid)) {
21            low = mid + 1;
22        } else if (key < dict.at(mid)) {
23            high = mid - 1;
24        } else { // key == dict.at(mid)
25            index = mid;
26            high = mid - 1; // keep searching for the leftmost key
27        }
28    }
29
30    if (index == -1) {
31        std::cout << "Not found!\n";
32    } else {
```

```

33     std::cout << std::format("The index of {} is {}.\n", key, index);
34 }
35
36 return 0;
37 }

```

样例输入 / 预期输出

```

1  1  =>  The index of 1 is 0.
2  34 =>  The index of 34 is 8.
3  0  =>  Not found!

```

`break` 语句不复存在是否会造成死循环呢？不会。这是因为，在 `while` 循环体中，不论 `if` 语句的哪个分支成立，要么 `low` 严格变大，要么 `high` 严格变小，区间 `[low, high]` 的长度不断缩短。

Line 17 处的循环条件 `low <= high` 总有一刻不再成立。

7.4 扩展：查找算法

查找 (Search) 算法 要求从给定结构中搜索特定的元素。待搜索的结构多种多样，查找算法也林林总总，其效率也各有不同。最简单的待搜索结构是无序数组。最简单的查找算法是顺序查找算法，它从头到尾依次扫描数组中的每个元素。假设数组的大小为 `n`，则在最坏情况下，顺序查找算法需要进行 `n` 次元素之间的比较操作。我们称其算法复杂度 (complexity) 是线性 (linear) 的。然而，当待搜索的结构为有序数组时，我们可以利用“有序”这个信息，大幅提高搜索的效率。



不难证明，对于无序数组，顺序查找算法是最优算法。你不妨试着证明一下。

数组是最简单的待搜索结构。更复杂的待搜索结构包括搜索树 (search tree)、哈希表 (hash table)、图 (graph) 等。其中，基于搜索树的搜索算法尤其丰富，涉及非常精彩的数据结构与算法知识。感兴趣的读者可以跟随 [MIT 的 Charles Leiserson & Erik Demaine 教授](#) 学习算法基础知识。

8. josephus

`n` 名士兵围成一圈，编号依次为从 `1` 到 `n`。从编号为 `1` 的士兵开始，每隔一名士兵杀掉下一名（杀第 `2` 名、隔第 `3` 名、杀第 `4` 名，……），绕圈继续，直到只剩下最后一名士兵。求最后剩下的士兵的编号。

这是经典约瑟夫环问题 ([Josephus problem @ Wikipedia](#))。

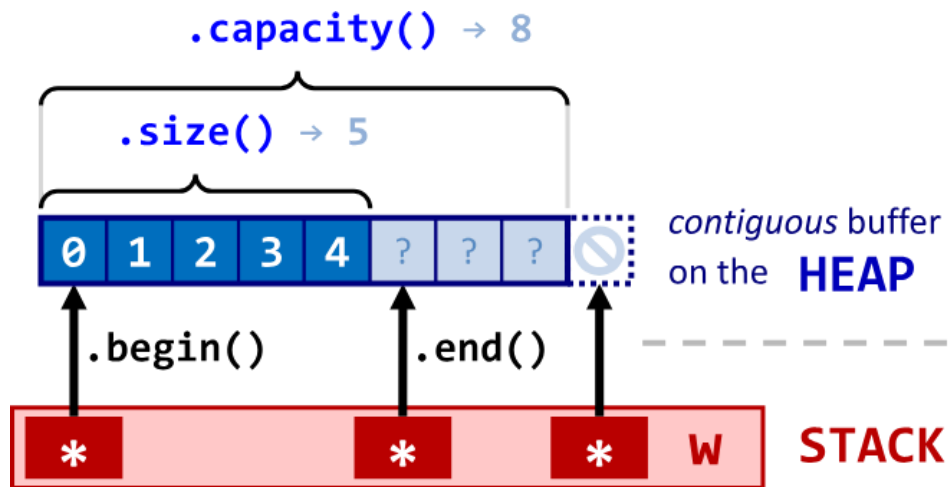
8.1 vector

首先，我们需要一个数据结构用于保存 `n` 名士兵的编号，而 `n` 的大小是由用户输入决定的。更重要的是，该数据结构要支持从中移除元素（士兵编号）。也就是说，该数据结构应是动态可调的。

`std::vector` @ [cppreference](#) 正是这样一种数据结构。

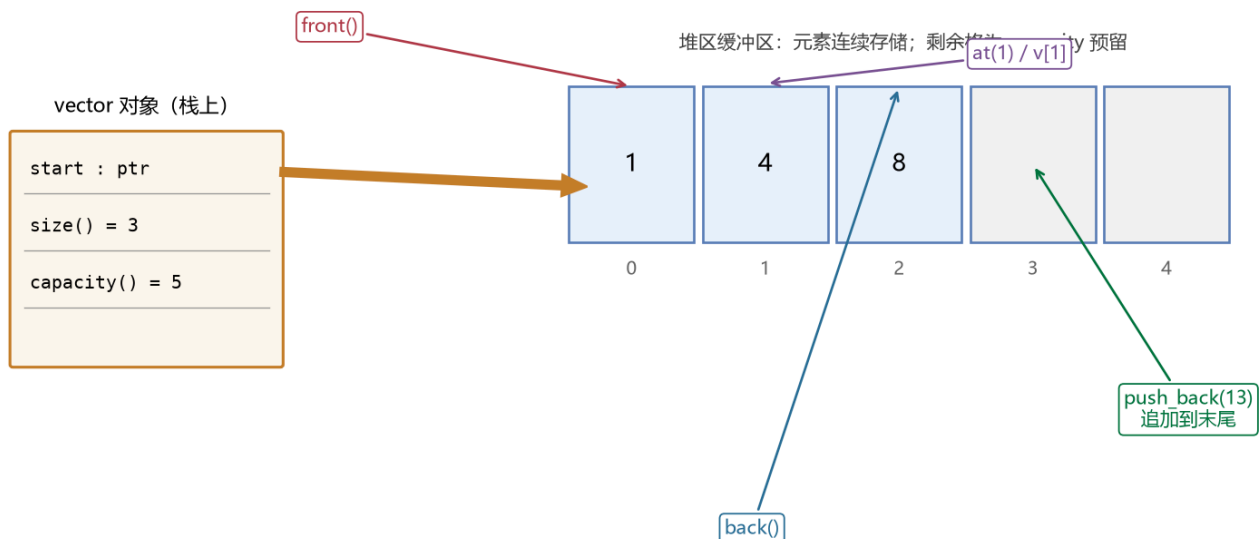
8.1.1 `vector` 中的数据

从概念上讲，`vector` 的底层实现就是一个数组，所有的元素在内存中**连续存储**。"连续存储"的特点非常重要，它决定了很多关键操作的性能，从而决定了 `vector` 的适用场景。



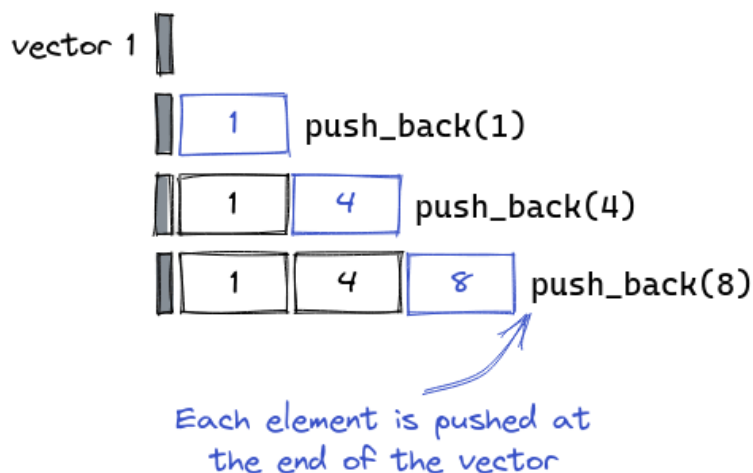
8.1.2 `vector` 上的操作

除了提供元素访问操作，它还封装了动态申请内存、动态增删元素的操作。下图集中展示了这些操作的位置关系：`front` / `back` 取两端元素，`v[i]` / `at(i)` 按下标访问，`push_back` 在末尾追加元素，`size` 是当前元素个数。



- `[]` (下标运算符)：按下标访问元素，**不检查越界**——`v[i]` 返回下标为 `i` 的元素（参见 [std::vector::operator\[\] @ cppreference](#)）；
- `at`：带越界检查的下标访问，越界时抛出 `std::out_of_range` 异常，便于发现程序错误（参见 [std::vector::at @ cppreference](#)）；
- `front`：返回第一个元素的引用（参见 [std::vector::front @ cppreference](#)）；

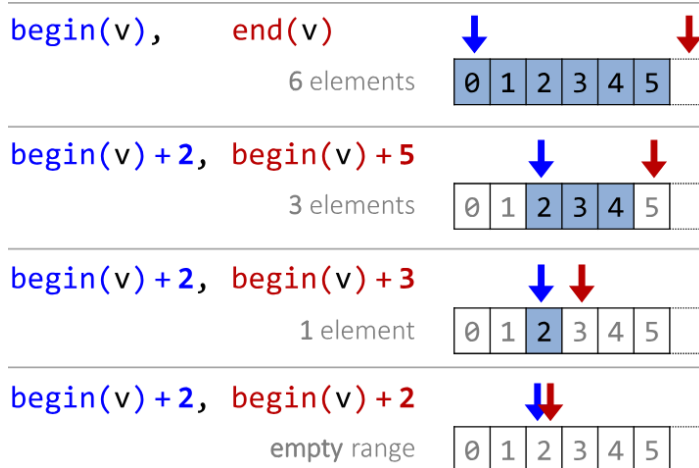
- `back`：返回最后一个元素的引用（参见 [std::vector::back @ cppreference](#)）；
- `size`：返回当前元素个数（参见 [std::vector::size @ cppreference](#)）；
- `push_back`：在末尾追加一个元素，必要时自动扩容（参见 [std::vector::push_back @ cppreference](#)）。



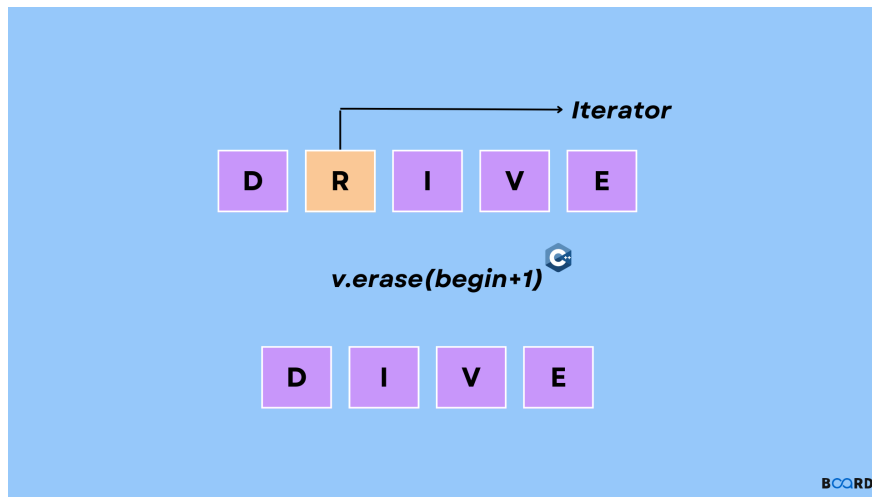
8.1.3 vector 上的迭代器

`vector` 还提供了迭代器 `iterator`，用于遍历其中的元素。考虑下图所示的 `v`。`begin(v)` 或 `v.begin()` 指向 `v` 的第一个元素，`end(v)` 或 `v.end()` 指向 `v` 的最后一个元素的下一个位置（注意，不是指向最后一个元素；具体原因，后续介绍）。通过 `begin + 2` 等算术运算可以移动迭代器位置，指向其它元素。

```
std::vector<int> v {0, 1, 2, 3, 4, 5};
```



- `erase`：删除指定位置的元素，后面的元素前移、`size()` 减 1（参见 [std::vector::erase @ cppreference](#)）。



8.2 `vector` as a kind of abstraction

`vector` 是一种容器（container）：用于容纳一组同类型元素、并提供统一访问与操作接口的数据结构。`vector` 是一种**数据抽象**——它把"连续内存、容量管理、元素移动"等实现细节隐藏起来，使用者只需面对"元素集合 + 增删改查"这一层语义，无需关心底层如何存储。

8.3 `josephus-vector-int`

josephus-vector-int.cpp

```
1  // File: josephus-vector-int.cpp
2  #include <cstdint>
3  #include <format>
4  #include <iostream>
5  #include <vector>
6
7  int main() {
8      int n{};
9      std::cin >> n;
10
11     // We assume that n >= 1.
12     // Store the numbers of the soldiers who are still alive.
13     std::vector<int> soldiers;
14
15     int number{1};
16     while (number <= n) {
17         soldiers.push_back(number);
18         ++number;
19     }
20
21     // The first soldier to be killed is soldier 2, which is at index 1.
22     std::size_t victim{1};
23
24     while (soldiers.size() > 1) {
25         soldiers.erase(soldiers.begin() + victim);
```

```

26
27     // After the erasure, the soldier now at victim is skipped,
28     // so the following soldier is killed next.
29     // The remainder operation wraps the index around the circle.
30     victim = (victim + 1) % soldiers.size();
31 }
32
33 std::cout << std::format("Last remaining soldier: {}\n", soldiers.front());
34
35 return 0;
36 }

```

样例输入 / 预期输出

```

1  5 => Last remaining soldier: 3
2  7 => Last remaining soldier: 7

```

首先，在 `while` 循环（第 15 到 19 行）中，使用 `push_back` 把 `1, 2, ..., n` 依次放入 `soldiers`。第 22 行的 `victim` 变量用于保存下一个被杀士兵在 `soldiers` 中的下标。注意，第一个被杀士兵的编号是 2，但其在 `soldiers` 中的下标是 1。

第 24 到 31 行是代码主体部分，模拟了整个残杀过程。第 25 行将被杀士兵从 `soldiers` 中移除。需要注意的是，`erase` 会导致 `victim` 下标后的士兵前移一位，这使得第 25 行执行后的 `victim` 指向了下一个幸存者，因此，第 30 行通过 `victim + 1` 指向下一个被杀士兵。如果 `victim + 1` 超出了当前 `soldiers` 的下标范围（0 到 `soldiers.size() - 1`），也就是等于 `soldiers.size()`，下一个被杀士兵的下标应该为 0，这就是第 30 行代码中 `% soldiers.size()` 的作用。该模拟过程持续进行，直到 `soldiers.size() == 1` 为止，见第 24 行。

`josephus-vector-int.cpp` 的关键操作是第 25 行的 `erase`。由于 `erase` 需要移动后面的元素，时间开销较大。如果 `n` 很大，`vector` 就不再适用了，此时可以考虑使用另一种数据结构——链表 [list @ cplusplus.com](#)，它支持常数时间内的元素插入与删除操作。

8.4 josephus-vector-bool

该方案中采用大小为 `n` 的 `vector<bool>` 保存每个人的存活状态，`true` 表示还活着，`false` 表示已被杀。

```

josephus-vector-bool.cpp
1  // File: josephus-vector-bool.cpp
2  #include <format>
3  #include <iostream>
4  #include <vector>
5

```

```

6  int main() {
7      int n{};
8      std::cin >> n;
9
10     // We assume that n >= 1.
11     // soldiers[i] indicates whether soldier i + 1 is still alive.
12     std::vector<bool> soldiers(n, true);
13
14     int remaining{n};
15
16     // Soldier 1, at index 0, is the first soldier to be skipped.
17     int survivor{0};
18
19     while (remaining > 1) {
20         // Find the next living soldier after survivor.
21         int victim{(survivor + 1) % n};
22         while (!soldiers.at(victim)) {
23             victim = (victim + 1) % n;
24         }
25
26         // Kill that victim.
27         soldiers.at(victim) = false;
28         --remaining;
29
30         // The next living soldier becomes the one to be skipped.
31         survivor = (victim + 1) % n;
32         while (!soldiers.at(survivor)) {
33             survivor = (survivor + 1) % n;
34         }
35     }
36
37     std::cout << std::format("Last remaining soldier: {}\n", survivor + 1);
38
39     return 0;
40 }

```

第 12 行 `std::vector<bool> soldiers(n, true);` 将 `soldiers` 中的 `n` 个元素均初始化为 `true`。变量 `survivor`（第 17 行）始终维护当前轮次中幸存者的下标，初始为 `0`；`victim`（第 21 行）始终维护当前轮次中受害者的下标。在同一轮中，`survivor` 与 `victim` 的关系是：
`victim == (survivor + 1) % n`。

由于受害者仍在 `soldiers` 中，其状态被设置为 `false`。因此，查找下一个受害者与幸存者时，都需要通过 `while` 循环跳过已经遇难的受害者，分别见第 21-24、31-34 行。

残忍的循环杀戮结束后，`survivor` 维护了最终幸存者的下标，`survivor + 1` 即为这位幸存者的编号（见第 37 行）。

8.5 josephus-formula

约瑟夫环问题也有数学解法：

如果 $n = 2^m + l$, $0 \leq l < 2^m$, 则答案为 $2l + 1$ 。

例如, 对于 $n = 12 = 2^3 + 4$, 幸存者编号为 $2 \times 4 + 1 = 9$ 。

josephus-formula.cpp

```
1 // File: josephus-formula.cpp
2 #include <bit>
3 #include <iostream>
4
5 int main() {
6     unsigned n{};
7     std::cin >> n;
8
9     unsigned highest_power{std::bit_floor(n)};
10    unsigned survivor{2 * (n - highest_power) + 1};
11
12    std::cout << "Survivor: " << survivor << '\n';
13 }
```

9. 小结

本章以 `while` 循环为主线, 通过逐步递进的示例, 系统介绍了循环的三要素 (条件、循环体、状态更新) 与循环不变式 (loop invariant) 的概念。与此同时, 还引入了 `array` 与 `vector` 两种容器类数据结构。另外, `break` 用于提前退出循环。接下来, [4-for【Being Updated】](#) 介绍 `for` 循环。

10. 修订记录

- 2026-09-18: 将第一章 (stars.c)、第二章 (primes.c)、第五章 (selection-sort.c) 移入《4-for》作为其第三、四、五章; 本文档剩余章节重新编号。
- 2026-09-18: 整合《3-while》教案示例, 章节重组为 digits-do-while.c、euclid.c、prime.c、palindrome.c、binary-search.c、josephus.c; 示例采用教学式渐进写法 (先给出有缺陷的版本, 逐步引入知识点改进)。
- 2026-09-21: 依据《AI 辅助写作准则》全面审阅修订: 新增「简介」章 (学习目标/重点/难点); binary-search 章节由 C 语言升级为 C++20 (`std::array`、`std::size_t`、`at()`, 文件统一为 `.cpp`); 全文代码内中文注释改为英文; 各示例补充样例输入/预期输出与代码块标题; 修正正文行号引用与括号笔误; 为相关知识点补充 cppreference/Wikipedia 链接。

- 2026-09-21: 章节重排——将 2.1 节“while 循环：数位之和”（含 `digits-sum.cpp`）移入《OJ Problems》备用；将 while 循环语法介绍（循环三要素）整合进 `euclid.c` 章节；将数位个数/ `do-while` /对比章节移至 `prime.c` 之后，突出 while 循环的教学主线。
- 2026-09-21: 新增 `fib.cpp` 章节（位于 `euclid.c` 之后）——用 `while` 循环计算第 `n` 个斐波那契数，定义采用英文 Wikipedia；先给出更新顺序有误的第一版，再引入临时变量修正，与 `euclid.c` 的更新顺序讲解相互呼应，并讨论边界情况、计数驱动与溢出。
- 2026-09-21: 依据《AI 写作准则》更新——各章节标题去除 `.c/.cpp` 后缀（如 `euclid.c` → `euclid`、`fib.cpp` → `fib`）；源代码文件名统一为 `.cpp` 后缀（如 `// File: euclid.cpp`）；代码块语言统一标注为 C++。
- 2026-09-23: 按《AI 写作准则》更新——`euclid` 章节补充朴素暴力版本 `euclid-bruteforce.cpp`（`while` 实现）并修正效率讨论；`digits 5.1` 改为“使用 `if` 特殊判定的正确 `while` 版本”，并由该复杂版本自然导出 `do-while`；`palindrome` 新增 6.5「`while` 初始化器」小节（保留不带初始化器版本，带初始化器版本作为进一步示范）；稍复杂输出改用 C++20 `std::format`（`fib` 首次引入并讲解）；所有 `.cpp` 代码及样例输出统一为英文；将 7.4「练习：优化 `log2.cpp`」移入《OJ Problems》备用。
- 2026-09-23: 将 `euclid-bruteforce` 小节移至 `euclid` 之后（重新编号为 2.5）；`euclid-bruteforce.cpp` 改用 `<algorithm>` 中的 `std::min` 初始化 `d`，并在代码讲解中首次介绍 `<algorithm>` 头文件与 `break` 语句（本讲第一次遇到 `break`）。
- 2026-09-23: `fib.cpp` 中将变量 `a`、`b`、`next` 重命名为 `pre_fib`、`cur_fib`、`next_fib`，并同步更新代码讲解。
- 2026-09-23: `prime` 章节四项更新——① `prime-bruteforce.cpp`、`prime-break.cpp`、`prime-sqrt.cpp` 补全为可编译运行的完整程序；② 明确 `i * i` 溢出属于有符号整数溢出的未定义行为（附 `cppreference` 依据），改用 `i <= n / i`；③ 讲解 `prime.cpp` 第 11 行 `return 0;` 的“提前返回”语义；④ 新增 4.5「性能分析」小节（实测数据、性能对比图与基准脚本 `prime_bench.cpp`）。
- 2026-09-23: 在 5.3「`while` 与 `do-while` 的对比」末尾补充 C++ Core Guidelines 的建议（ES.75: Avoid `do` -statements——优先使用 `while`），并说明可读性与防错两方面的理由。
- 2026-09-23: `binary-search.cpp` 中 `index` 的哨兵值由 `dict.size()` 改为 `-1`（更直观、且符合常见查找接口惯例），类型相应由 `std::size_t` 改为带符号的 `int`，并同步更新代码注释与 `Line 15`、`Line 32` 的讲解。
- 2026-09-23: 将第 4 节 `prime`（含 4.5「性能分析」及附图与基准脚本）整体移入《4-for【Being Updated】》末尾备用；本文档剩余章节重新编号。

- 2026-09-23: 依据《AI Writing Guidelines》全面审阅并完善——①修正 `binary-search` 章节代码讲解的行号引用 (`low/high` 声明 → `Line 13/Line 14`, `index` 声明 → `Line 16`, 循环条件 → `Line 17`, `index = mid` → `Line 28`, `if (index == -1)` → `Line 33`, `break` → `Line 29`) ; ② `binary-search-leftmost.cpp` 与“向左扫描的朴素做法”片段改用 `-1` 哨兵, 与 `binary-search.cpp` 保持一致; ③ `binary-search.cpp` 代码注释中的章节引用由 7.2.3 修正为 6.2.3; ④ `fib` 讲解“数据驱动”示例由已移出的 `prime` 改为 `euclid`、`binary-search`; ⑤ `digits-do-while` 4.1 “5.2 节”修正为“4.2 节”; ⑥ `euclid-bruteforce` 小节补充样例输入/预期输出; ⑦简介难点条目措辞修正; ⑧经权威核验 (`cppreference while` 页), C++ 标准不支持 `while` 初始化器 (`init-statement`) 语法, 故不引入此前规划的 `palindrome while` 初始化器示例。
- 2026-09-23: 在第 2 节末尾新增 2.6 `euclid-while-decl` ——介绍 `while` 的条件可以是简单声明 (决策变量 `decision variable`, 引用 `cppreference while` 页原文并给出 `expressions / declarations` 链接); 将 `euclid-bruteforce.cpp` 中 `d` 的声明移入条件、并把 `cout` 行移至 `break` 前; 实测验证 (GCC 16, `-std=c++20`) 该写法因条件声明每轮重建而陷入死循环, 与 `while (d >= 1)` 在语句层面并不等价 (仅条件真值等价); 对比 `while (int d{...}; d >= 1)` 的语法错误 (`while` 没有初始化器, 仅 `if / switch` 自 C++17 起支持), 并解释原因: `for` 已覆盖初始化器需求。
- 2026-09-23: `binary-search.cpp` 移除循环中的 `if (mid == 0)` 特判, `low/high/mid` 由 `std::size_t` 改为带符号 `int` (`high = mid - 1` 在 `mid` 为 0 时变为 `-1`, 循环条件自然失效), 更贴近二分查找的标准写法 (对应 CG ES.102); `binary-search-leftmost.cpp` 同步简化 (找到 `key` 后以 `high = mid - 1` 继续向左搜索, 无需 `mid == 0` 特判); 同步更新 6.2.2 代码讲解 (行号与类型说明)、6.2.3 与 6.3 的讲解; 代码经 GCC 16 (`-std=c++20`) 编译并实测 (含边界 `55`、`100`) 通过。
- 2026-09-24: 依据《AI Writing Guidelines》4.1「完整代码须可编译、可运行」全面审阅全文 .cpp 代码——用本机 GCC 16.2 (`-std=c++20 -Wall -Wextra`) 逐一编译并运行全部完整程序 (`euclid`、`euclid-bruteforce`、`fib`、`digits-while`、`digits-do-while`、`palindrome`、`binary-search`、`binary-search-leftmost`、`josephus`), 文档所列 21 组样例输入/预期输出与 16 组补充边界输入 (数组两端与越界、负数、单元素、`n = 1`、`F(46)` 上限等) 全部一致; 2.6 节两个明确标注的错误示例 (`euclid-while-decl-wrong.cpp` 编译合法但运行死循环、`while` 初始化器语法错误片段) 符合 4.1 豁免条件, 予以保留; 未发现需修订的代码。
- 2026-09-24: `fib` 章节更新——①代码改为用户修订版 (变量 `pre_fib/cur_fib/next_fib/count` 重命名为 `previous/current/next/index`, 缩进与文件头注释相应调整, 代码经 GCC 16.2 (`-std=c++20`) 编译并实测 `0/1/7/46` 通过); ②3.1 识别 loop 与 3.2 代码讲解中的变量名、行文全部同步; ③正式引入循环不变式概念 (loop invariant): `fib.cpp` 的不变式为 “At the beginning of each iteration, `previous == F(index)` and `current == F(index + 1)`”, 并按“初始化—保持—终止”三段验证循环正确性。

- 2026-09-24: 依据旧版《3-while》第八节「循环不变式」高度凝练编写三节内容——①2.2 循环不变式: 补充「不变的是关系而非变量值」的澄清、引导问题 (What remains true before every iteration?) 与对 3.4/4.3 实例的预告; ②2.3 循环不变式作为一种抽象: 阐述语义抽象 (discard irrelevant detail, preserve relevant structure), 以 `euclid` 的具体状态序列为例, 归纳状态抽象/时间抽象/推理抽象三个层面, 并与函数抽象、三阶段发现法相呼应; ③3.4 Loop Invariant (`euclid`): 给出核心不变式 $\text{gcd}(a, b) = \text{gcd}(a_0, b_0)$, 按初始化—保持—终止三步验证, 补充终止性论证 (`b` 严格递减) 与 `48 18` 实例。
- 2026-09-24: `binary-search` 章节重构为“先 C 传统数组、后 `std::array`”的教学结构——①7.2.2 `binary-search`: 代码改为 C 传统数组版本 (`const int dict[]{...}`、`dict[mid]` 下标访问、`sizeof(dict) / sizeof(dict[0])` 求长度), 讲解中新增 C 数组语法介绍 (声明/初始化列表推导长度/下标从 0 开始/越界为未定义行为), 行号引用全部同步; ②7.2.3 `binary-search-array`: 保留 `std::array` 版本并命名为 `binary-search-array.cpp`, 新增 `std::array` 介绍 (定长、`.size()`、`.at()` 越界检查、`constexpr`、不退化), 附 `cppreference` 链接 (`std::array / std::array::size / std::array::at`), 并与 C 数组版本逐点对照; 两个版本代码均经 GCC 16.2 (`-std=c++20`) 编译并实测 (`34 / 1 / 0 / 55 / 100 / -1 / 2`) 通过。
- 2026-09-24: 将第 8 节 `josephus` (8.1 问题分析/8.2 引入 `std::vector` / 8.3 模拟过程, 含 `josephus.cpp`、样例输入与讲解) 整体移入《5-loops 【Being Updated】》末尾 (附录前) 备用; 本章新增第 8 节「小结」, 凝练回顾: `while` 语法与语义 (含 decision variable、`do-while` 对比与 CG ES.75)、循环不变式 (`euclid` / `fib` 实例与“初始化—保持—终止”验证)、计数驱动与条件驱动两种遍历形态、`break` 提前退出、数据组织演进 (单变量 → C 数组 / `std::array` → `std::vector`) 与工程细节 (`std::format`、带符号下标与 `-1` 哨兵、`.at()` 越界检查)。
- 2026-09-27: `josephus` 章节改写为“隔一人杀一人” (`k = 2` 特化, 去掉报数描述)——①8.1 问题分析改为“隔一人杀一人”表述 (杀第 2 个、隔第 3 个、杀第 4 个……), 并说明其与经典约瑟夫环问题 `k = 2` 情形等价; ②8.2 标记法 (`vector<bool>`): `josephus.cpp` 直接实现“隔一人杀一人”, 讲解被杀人占位、必须跳过死人 (代码中两处 `while` 循环) 的缺陷; ③8.3 改进: 直接移除 (`vector<int>` 与 `erase`)——回答“为什么不用 `vector<int>` 存编号、将被杀人直接移除”: 用 `while` 循环 `push_back` 填充 `1..n`, `erase` 移除被杀者, 讲解 `erase` 后 `index` 自动指向新一轮起点、容器大小即圈内人数 (附 `push_back` / `erase` 的 `cppreference` 链接), 并与标记法对比权衡; 两个版本代码均经 GCC 16.2 (`-std=c++20`) 编译并实测 (`5 / 7 / 1 / 2`) 通过。

- 2026-09-27: 依据《AI Writing Guidelines》全面审阅并修订——①全文代码块缩进统一为 2 空格（`euclid`、`euclid-bruteforce`、`euclid-while-decl-wrong`、`digits-while`、`digits-do-while`、`palindrome`、`binary-search`、`binary-search-array`、`binary-search-leftmost`、向左扫描朴素做法片段、错误示例片段），代码逻辑与行为不变，全部经本机 GCC 16.2（`-std=c++20 -Wall -Wextra`）重新编译并实测通过；②删除 7.2.2 节关于 `mid` 溢出防护的 callout（代码保持 $(low + high) / 2$ 不变，避免正文与代码不一致）；③7.2.3 讲解中 `constexpr` 统一为 `const`（与代码 `const std::array<int, 10>` 一致），`binary-search-leftmost.cpp` 的 `constexpr` 同步改为 `const`；④7.3 向左扫描朴素做法片段由 `dict.at(index - 1)` 改为 `dict[index - 1]`（C 数组语法），行号引用修正为 `Lines 23--26 / Line 25`；⑤8.3 讲解中 `soliders` 拼写修正为 `soldiers`；⑥8.1/8.3 的 `vector` / `list` 链接改为 `/w/cpp/container/...` 规范路径，链接文字统一为“`std::vector @ cplusplusreference`”格式；⑦8.2 续写为完整段落，从数据抽象（data abstraction）的角度解释 `vector` 为何是一种抽象；⑧8.1.2 操作列表（`[]`、`at`、`front`、`back`、`size`、`push_back`）逐一补充 `cplusplusreference` 链接与简要介绍，新增自绘示意图（`vector` 栈对象 + 堆区缓冲区 + 各操作位置标注）；8.1.3 的 `erase` 条目同步补链接与说明；⑨8.5 节内容非空，未做改动；⑩全文统一“`xxx` 循环”书写（中文与代码之间不留空格）。
- 2026-09-27: 将《4-for》第 1.1 节 `array` 的 C 传统数组详细介绍整合进 7.1——7.1.1 补充数组类型含大小信息（`int [3]` 与 `int [5]` 是不同类型）、列表初始化补零规则、空花括号值初始化、省略大小由编译器推导；7.1.2 补充下标即偏移量、`sizeof` 求数组大小及括号/编译期语义。仅涉及 C 传统数组，不涉及 `std::array`。

（注：部分内容由豆包工作 AI 生成）